Note:

Click here to download the full example code

Calculating the Pearson correlation coefficient between datasets

In this recipe, we will take two datasets, one for an independent variable (in this example elevation) and one for a dependent variable (snow cover over a particular day), regrid them to the same resolution then calculate the correlation coefficient, to get a measure of the relationship between them.

1. Import cf-python, cf-plot and other required packages:

```
import matplotlib.pyplot as plt
import scipy.stats.mstats as mstats
import cfplot as cfp

import cf
```

2. Read the data in and unpack the Fields from FieldLists using indexing. In our example We are investigating the influence of the land height on the snow cover extent, so snow cover is the dependent variable. The snow cover data is the 'Snow Cover Extent 2017-present (raster 500 m), Europe, daily – version 1' sourced from the Copernicus Land Monitoring Service which is described at: https://land.copernicus.eu/en/products/snow/snow-cover-extent-europe-v1-0-500m and the elevation data is the 'NOAA NGDC GLOBE topo: elevation data' dataset which can be sourced from the IRI Data Library, or details found, at: http://iridl.ldeo.columbia.edu/SOURCES/.NOAA/.NGDC/.GLOBE/.topo/index.html.

```
orog = cf.read("~/recipes/1km_elevation.nc")[0]
snow = cf.read("~/recipes/snowcover")[0]
```

3. Choose the day of pre-aggregated snow cover to investigate. We will take the first datetime element corresponding to the first day from the datasets, 1st January 2024, but by changing the indexing you can

explore other days by changing the index. We also get the string corresponding to the date, to reference later:

```
snow_day = snow[0]
snow_day_dt = snow_day.coordinate("time")[0].data
snow_day_daystring = f"{snow_day_dt.datetime_as_string[0].split(' ')[0]
```

4. Choose the region to consider to compare the relationship across, which must be defined across both datasets, though not necessarily on the same grid since we regrid to the same grid next and subspace to the same area for both datasets ready for comparison in the next steps. By changing the latitude and longitude points in the tuple below, you can change the area that is used:

```
region_in_mid_uk = ((-3.0, -1.0), (52.0, 55.0))
sub_orog = orog.subspace(
    longitude=cf.wi(*region_in_mid_uk[0]), latitude=cf.wi(*region_in_mi
)
sub_snow = snow_day.subspace(
    longitude=cf.wi(*region_in_mid_uk[0]), latitude=cf.wi(*region_in_mi
)
```

5. Ensure data quality, since the standard name here corresponds to a unitless fraction, but the values are in the tens, so we need to normalise these to all lie between 0 and 1 and change the units appropriately:

```
sub_snow = (sub_snow - sub_snow.minimum()) / (sub_snow.range())
sub_snow.override_units("1", inplace=True)
```

6. Regrid the data so that they lie on the same grid and therefore each array structure has values with corresponding geospatial points that can be statistically compared. Here the elevation field is regridded to the snow field since the snow is higher-resolution, but the other way round is possible by switching the field order:

```
regridded_orog = sub_orog.regrids(sub_snow, method="linear")
```

7. Squeeze the snow data to remove the size 1 axes so we have arrays of the same dimensions for each of the two fields to compare:

```
sub_snow = sub_snow.squeeze()
```

8. Finally, perform the statistical calculation by using the SciPy method to find the Pearson correlation coefficient for the two arrays now they are in comparable form. Note we need to use ‘scipy.stats.mstats’ and

not ‘scipy.stats’ for the ‘pearsonr’ method, to account for masked data in the array(s) properly:

```
coefficient = mstats.pearsonr(regridded_orog.array, sub_snow.array)
print(f"The Pearson correlation coefficient is: {coefficient}")
```

Out:

```
The Pearson correlation coefficient is: PearsonRResult(statistic=np.
```

9. Make a final plot showing the two arrays side-by-side and quoting the determined Pearson correlation coefficient to illustrate the relationship and its strength visually. We use ‘gpos’ to position the plots in two columns and apply some specific axes ticks and labels for clarity.

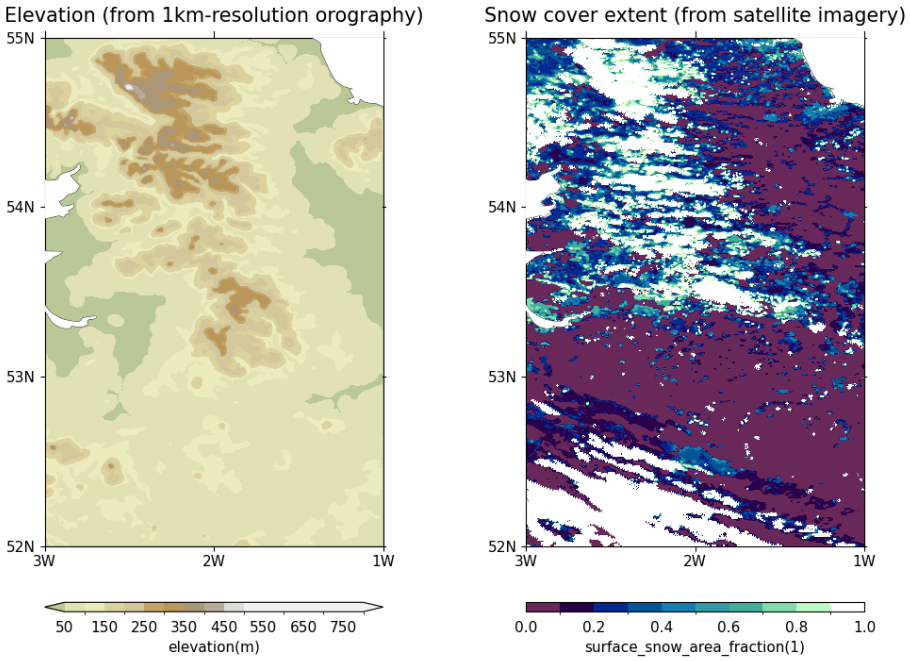
```
cfp.gopen(
    rows=1,
    columns=2,
    top=0.85,
    user_position=True,
)

# Joint configuration of the plots, including adding an overall title
plt.suptitle(
    (
        "Snow cover compared to elevation for the same area of the UK "
        f"aggregated across\n day {snow_day_daystring} with correlation
        "coefficient (on the same grid) of "
        f"{coefficient.statistic:.4g} (4 s.f.)"
    ),
    fontsize=17,
)
cfp.mapset(resolution="10m")
cfp.setvars(ocean_color="white", lake_color="white")
label_info = {
    "xticklabels": ("3W", "2W", "1W"),
    "yticklabels": ("52N", "53N", "54N", "55N"),
    "xticks": (-3, -2, -1),
    "yticks": (52, 53, 54, 55),
}

# Plot the two contour plots as columns
cfp.gpos(1)
cfp.cscale("wiki_2_0_reduced", ncols=11)
cfp.con(
    regridded_orog,
    lines=False,
    title="Elevation (from 1km-resolution orography)",
    colorbar_drawedges=False,
    **label_info,
)
cfp.gpos(2)
# Don't add extentions on the colourbar since it can only be 0 to 1 inc
cfp.levs(min=0, max=1, step=0.1, extend="neither")
cfp.cscale("precip_11lev", ncols=11, reverse=1)
cfp.con(
    sub_snow,
```

```
    lines=False,
    title="Snow cover extent (from satellite imagery)",
    colorbar_drawedges=False,
    **label_info,
)
```

Snow cover compared to elevation for the same area of the UK aggregated across day 2024-01-01 with correlation coefficient (on the same grid) of 0.3587 (4 s.f.)



Total running time of the script: (0 minutes 16.503 seconds)

Download Python source code: plot_18_recipe.py

Download Jupyter notebook: plot_18_recipe.ipynb

Gallery generated by Sphinx-Gallery